

WEB APPLICATION PENETRATION TESTING REPORT

Comprehensive Security Assessment of a Ride-Sharing Web Application

Prepared By: CH. Sai Varun

Role: Cybersecurity Enthusiast | Application Security | OWASP Testing

Application Name: RideConnect (Ride-Sharing Web Application)

Application Type: Web Application

Testing Environment: Local Development (Authorized Testing)

Frontend: React

Backend: Spring Boot

Database: MySQL

Authentication: JSON Web Token (JWT)

Testing Methodology: OWASP Web Security Testing Guide (WSTG) & OWASP Top 10 (2021)

Testing Tool: Burp Suite Community Edition

Total Test Cases Executed: 48

Report Version: 1.0

Date: June 2026

CONFIDENTIALITY STATEMENT

This report documents the results of an authorized security assessment performed exclusively against a locally hosted web application developed and owned by the author. All testing activities were conducted in a controlled development environment with full authorization. This report is intended for educational purposes, secure software development, and professional portfolio demonstration.

No testing was performed against third-party or production systems. Sensitive identifiers (full tokens, credentials, internal hostnames) have been partially redacted or replaced with placeholders for public sharing.

Table of Contents

3. Executive Summary.....	6
4. Scope of Assessment.....	8
5. Methodology (OWASP WSTG).....	9
6. Testing Environment.....	11
7. Risk Rating Methodology (CVSS).....	12
8. Executive Dashboard.....	13
9. Vulnerability Summary.....	14
10. Detailed Findings (48 Test Cases).....	15
Test 1 — Application Reconnaissance & Mapping.....	15
Test 11 — Driver Profile IDOR.....	22
Test 21 — Rider Identity Tampering (IDOR).....	30
Test 24 — Fare Manipulation.....	33
Test 31 — Mass Assignment.....	37
Test 48 — Sensitive Information Disclosure.....	49
11. OWASP Top 10 (2021) Mapping.....	50
12. Secure Coding Recommendations.....	52
13. Overall Security Score.....	56
14. Conclusion.....	57
15. Appendix.....	58
Appendix A — Tools Used.....	58
Appendix B — Testing Environment.....	58
Appendix C — Burp Suite Workflow.....	58
Appendix D — Payloads Used.....	59
Appendix E — HTTP Status Codes Observed.....	59
Appendix F — Burp Suite Features Utilized.....	60
Appendix G — Screenshot Index.....	60
Appendix H — References.....	61
Appendix I — Authorization Statement.....	61

List of Figures

Figures referenced throughout this report consist of Burp Suite Proxy, Repeater, Intruder, and Decoder screenshots captured directly during testing. Each figure is numbered according to its associated Test Case (e.g., Figure — Test 11) and is presented inline within Section 10 (Detailed Findings). A consolidated index of all screenshots is provided in Appendix G.

List of Tables

- Table 1 — Executive Dashboard: Test Outcome Summary
- Table 2 — Vulnerability Summary by Severity
- Table 3 — CVSS v3.1 Severity Rating Scale
- Table 4 — Detailed Findings Summary (All 48 Tests)
- Table 5 — OWASP Top 10 (2021) Mapping
- Table 6 — Overall Security Score Calculation
- Table 7 — Burp Suite Features Utilized
- Table 8 — HTTP Status Codes Observed
- Table 9 — Screenshot Index

3. Executive Summary

This report presents the findings of an authorized, manual penetration test performed against RideConnect, a ride-sharing web application built on a React frontend, a Spring Boot REST API backend, and a MySQL database, secured with JSON Web Token (JWT) based authentication. The assessment was performed entirely against a local development instance owned by the author.

Testing followed the OWASP Web Security Testing Guide (WSTG) and was structured around the OWASP Top 10 (2021) risk categories, using Burp Suite Community Edition as the primary testing platform (Proxy, Repeater, Intruder, and Decoder).

Total Test Cases Executed: 48

Tests Passed (Secure Behavior Confirmed): 31 (64.6%)

Tests Failed (Vulnerability Confirmed): 13

Informational Observations: 4

Unique Vulnerabilities Identified: 16 (after deduplicating findings validated through multiple related test cases)

1.1 Key Findings

The assessment confirmed that RideConnect's authentication layer — including JWT signature verification, token expiration handling, malformed/missing token rejection, OTP enforcement, and SQL Injection / Cross-Site Scripting defenses — is implemented soundly. However, the assessment identified a systemic weakness in object-level authorization: multiple endpoints trust client-supplied identifiers (driver IDs, rider IDs, ride IDs) instead of deriving identity from the authenticated JWT. This pattern produced 2 Critical and 5 High severity findings, the majority rooted in Broken Access Control (OWASP A01).

Additional findings include exposure of BCrypt password hashes in API responses (OWASP A02), a client-controlled fare calculation flaw (OWASP A04), and several security-misconfiguration and information-disclosure observations (OWASP A05) that, while lower in severity individually, reduce the overall defense-in-depth posture of the application.

1.2 Overall Risk Rating

Overall Security Score	20 / 100 (Grade F — Critical Risk)
Test Pass Rate	64.6% (31 of 48 tests confirmed secure)

This dual-metric approach is intentional: the raw pass rate (64.6%) reflects that most individual controls tested were sound, while the severity-weighted Security Score reflects that the Critical and High findings present a realistic path to account takeover and financial abuse if left unremediated — and therefore dominate the overall risk picture.

1.3 Top Recommendations

- Derive resource ownership from the authenticated JWT on every endpoint — never trust a client-supplied numeric ID (resolves all IDOR/BOLA findings).
- Remove the password field from all API serialization paths using dedicated response DTOs.
- Calculate ride fares exclusively server-side; never accept fare as client input.

- Replace direct entity binding with allow-listed DTOs on all write endpoints to eliminate mass-assignment risk.
- Implement authentication rate-limiting / progressive logout.
- Add a restrictive Content-Security-Policy header and tighten verbose error responses.

4. Scope of Assessment

2.1 In-Scope Components

- React frontend (Rider and Driver web portals) — <http://127.0.0.1:5173>
- Spring Boot REST API backend — <http://localhost:8081> / :8082
- MySQL-backed data layer (assessed indirectly via the API)
- JWT-based authentication and authorization mechanisms
- Ride booking, scheduling, acceptance, and fare-calculation business logic
- Driver and rider profile management, including file/avatar upload
- HTTP security headers, CORS configuration, and session/cache handling

2.2 Out-of-Scope Components

- Third-party payment processor infrastructure (Stripe) — only the application's integration points were observed in transit, not Stripe's own systems
- Denial-of-Service (DoS) / load testing
- Physical security and social engineering
- Mobile application clients (none were in use; testing was web-only)

2.3 Rules of Engagement

All testing was conducted by the application's author/developer against a self-hosted local instance, with full knowledge of the underlying architecture (gray-box conditions were used for some tests, e.g., JWT structure analysis, while black-box conditions were used for others, e.g., initial reconnaissance). No production systems, third-party infrastructure, or other users' data were accessed at any point.

5. Methodology (OWASP WSTG)

Testing followed the structure of the OWASP Web Security Testing Guide (WSTG), supplemented by the OWASP Top 10 (2021) risk taxonomy and the OWASP API Security Top 10 (2023) for API-specific findings. The assessment was carried out in six phases:

Phase 1 — Reconnaissance

- Configure Burp Suite Proxy
- Intercept application traffic
- Map application endpoints
- Identify authentication flow

Phase 2 — Authentication Testing

- Login validation
- JWT extraction
- JWT decoding
- Token tampering
- Missing token validation
- Expired token validation

Phase 3 — Authorization Testing

- IDOR / BOLA
- Role validation
- Resource ownership verification
- Business logic testing

Phase 4 — Input Validation

- SQL Injection
- Cross-Site Scripting (Stored & Reflected)
- HTTP Parameter Pollution
- HTTP Method Tampering
- Mass Assignment

Phase 5 — Secure Configuration

- Security headers
- CORS
- TRACE method
- Directory Traversal
- Host Header Injection
- Cache Control
- Error Handling

Phase 6 — Reporting

- Vulnerability verification
- Severity assessment (CVSS v3.1)
- Evidence collection (screenshots, request/response pairs)
- Remediation recommendations

3.1 Burp Suite Workflow

The practical workflow followed for nearly every test case was:

Recon → Proxy → Repeater → Intruder → Manual Verification → Evidence Capture → Report

6. Testing Environment

Component	Technology / Value
Frontend	React (Vite dev server)
Backend	Spring Boot REST API
Database	MySQL
Authentication	JSON Web Token (JWT), HS512 signing
File Storage	Local storage (avatar uploads)
Testing Environment	Localhost
Frontend URL	http://127.0.0.1:5173
Backend URL	http://localhost:8081 (also referenced as :8082 in earlier sessions)
Real-time Channel	WebSocket (ride status / notifications)
Payment Integration	Stripe (test mode)

7. Risk Rating Methodology (CVSS)

Each confirmed finding was assigned a qualitative severity rating aligned to the Common Vulnerability Scoring System (CVSS) v3.1 severity bands, as shown below. Where a precise CVSS vector was constructed, it is presented alongside the relevant finding.

Severity	CVSS v3.1 Score Range	Risk Implication
Critical	9.0 – 10.0	Immediate, severe impact; remediate before any deployment
High	7.0 – 8.9	Significant impact; remediate as top priority
Medium	4.0 – 6.9	Moderate impact; remediate in near-term release cycle
Low	0.1 – 3.9	Minor impact; remediate as part of routine hardening
Informational	0.0	No direct exploitability; defense-in-depth improvement

5.1 Scoring Approach Used in This Report

- Critical: vulnerabilities enabling direct impersonation of another user or full takeover of another user's transactions (e.g., Rider Identity Tampering, Unauthorized Ride Acceptance).
- High: vulnerabilities allowing unauthorized read/write access to another account's data, or direct manipulation of financial values (e.g., IDOR findings, Mass Assignment, Fare Manipulation).
- Medium: vulnerabilities that increase attacker capability or reduce defense-in-depth without directly compromising a specific account on their own (e.g., missing rate limiting, password hash exposure, weak file content validation).
- Low / Informational: configuration hardening opportunities and information disclosure that aid reconnaissance but are not independently exploitable (e.g., missing CSP, verbose error pages).

8. Executive Dashboard

Table 1 — Test Outcome Summary

Outcome	Count	% of Total
Total Test Cases Executed	48	100%
PASS (Secure Behavior Confirmed)	31	64.6%
FAIL (Vulnerability Confirmed)	13	27.1%
INFORMATIONAL	4	8.3%

Unique Findings by Severity

Critical	2
High	5
Medium	6
Low / Informational	3
Total Unique Findings	16

9. Vulnerability Summary

A total of 16 unique vulnerabilities were identified across 17 failing or informational test executions (some vulnerabilities were independently validated through more than one related test case — for example, password hash exposure was confirmed via Tests 10, 12, and 48).

Vulnerability	Severity	Test ID(s)	Status
Broken Access Control – Insecure Direct Object Reference (IDOR)	Critical	T21	Identified
Broken Access Control (IDOR) – Unauthorized Ride Acceptance	Critical	T22	Identified
Insecure Direct Object Reference (IDOR)	High	T11	Identified
Broken Access Control (IDOR) – Driver Scheduled Ride Disclosure	High	T13	Identified
Broken Access Control	High	T14	Identified
Business Logic Vulnerability	High	T24	Identified
Mass Assignment Vulnerability	High	T31	Identified
Missing Login Rate Limiting	Medium	T5	Identified
Sensitive Information Exposure – Password Hash Disclosure	Medium	T10	Identified
Sensitive Information Disclosure	Medium	T12	Identified
Weak File Content Validation	Medium	T18	Identified
Missing Rate Limiting	Medium	T25	Identified
Sensitive Information Disclosure – Password Hash Exposure	Medium	T48	Identified
Information Disclosure through Verbose Error Messages	Low	T44	Identified
Improper HTTP Status Code Handling	Low	T47	Identified
Missing Content Security Policy	Informational	T40, T46	Identified

10. Detailed Findings (48 Test Cases)

Each test case below follows a consistent structure: Objective, Tools Used, Steps Performed, Evidence, Observed Result, Analysis, Finding (where applicable), Severity, Remediation, and Status. Screenshots are embedded directly beneath the relevant test where Burp Suite capture evidence was preserved; for tests without an embedded image, evidence is described textually and the corresponding raw capture is indexed in Appendix G.

Test 1 — Application Reconnaissance & Mapping

OWASP Category	Severity	Status
N/A (Reconnaissance)	None	PASS

Objective: Identify the application's attack surface, enumerate accessible endpoints, understand the application architecture, and map the communication between the frontend and backend.

Tools Used: Burp Suite Proxy, Browser Developer Tools, HTTP History

Steps Performed:

- Configured Burp Suite as the browser proxy.
- Opened the React application running on <http://127.0.0.1:5173>.
- Logged in as both Driver and Rider to generate application traffic.
- Intercepted HTTP requests using Burp Proxy.
- Reviewed HTTP History to identify backend endpoints.
- Documented authentication, profile, ride management, and upload endpoints.

Evidence: Observed endpoints included: POST /api/drivers/login, POST /api/users/login, GET /api/profile/{id}, PUT /api/profile/{id}, POST /api/rides/book-scheduled, POST /api/rides/{id}/accept, GET /api/rides, PUT /api/driver/status

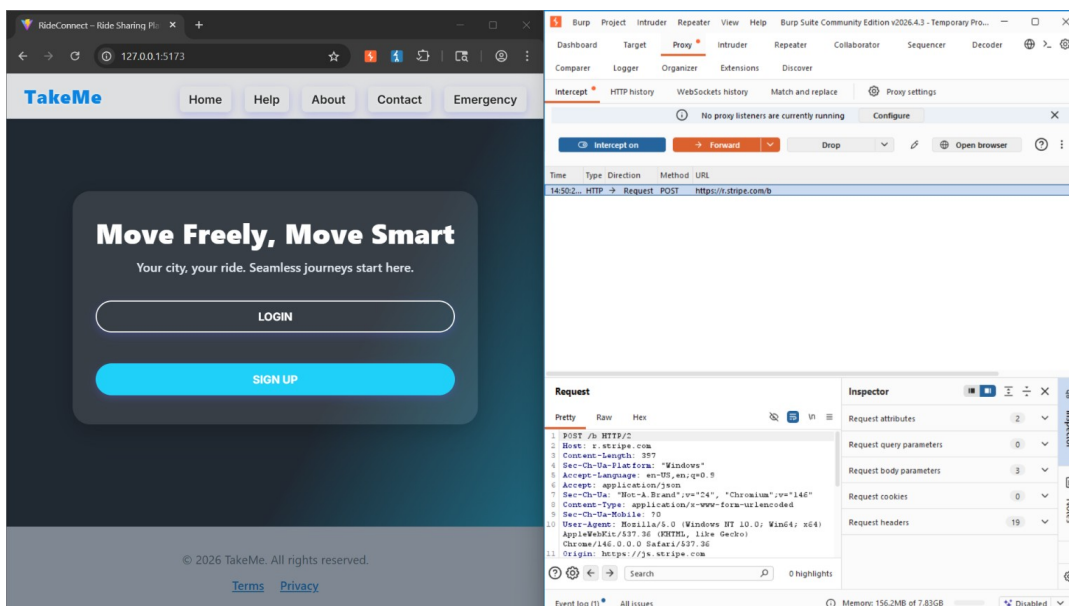


Figure — Test 1: Application Reconnaissance & Mapping (Burp Suite capture)

Observed Result: The application exposes REST APIs over HTTP between the React frontend and the Spring Boot backend. JWT Bearer tokens are required for authenticated endpoints.

Analysis: The application architecture was successfully mapped. No unexpected or hidden endpoints were identified during initial reconnaissance.

Remediation:

- Maintain API documentation and restrict unused endpoints in production deployments.

Status: **PASS**

Test 2 — Authentication Testing

OWASP Category	Severity	Status
A07 – Identification & Authentication Failures	None	PASS

Objective: Verify that only users with valid credentials can successfully authenticate.

Tools Used: Burp Proxy, Burp Repeater

Steps Performed:

- Captured the login request.
- Sent valid credentials.
- Sent invalid credentials.
- Observed authentication responses.

Evidence: Endpoints tested: POST /api/drivers/login, POST /api/users/login

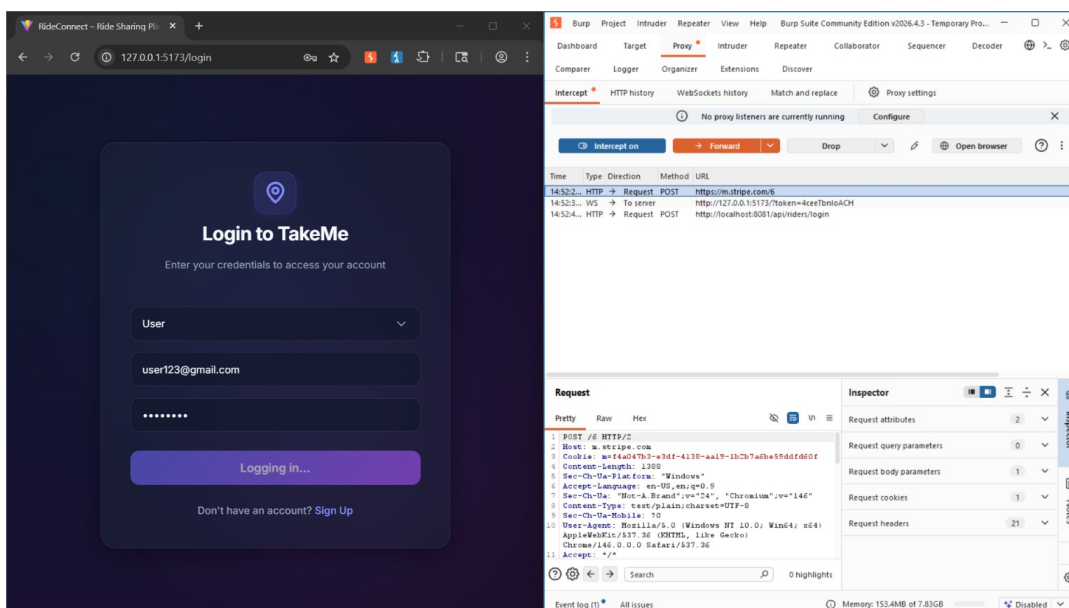


Figure — Test 2: Authentication Testing (Burp Suite capture)

Observed Result: Valid credentials generated a JWT. Invalid credentials returned authentication errors. Login bypass attempts were unsuccessful.

Analysis: Authentication is correctly implemented. Unauthorized users cannot authenticate using invalid credentials.

Remediation:

- Continue enforcing JWT authentication and secure password storage.

Status: **PASS**

Test 3 — JWT Authentication Validation

OWASP Category	Severity	Status
A07 – Identification & Authentication Failures	None	PASS

Objective: Verify that protected endpoints require a valid JWT token.

Tools Used: Burp Repeater

Steps Performed:

- Removed the Authorization header.
- Sent requests without JWT.
- Sent malformed JWT.
- Sent expired JWT.
- Modified JWT payload without updating the signature.

Evidence: Example request: GET /api/profile/3 with Authorization header removed.

Observed Result: The server rejected: missing JWT, invalid JWT, expired JWT, and modified JWT.

Analysis: The application correctly validates JWT signatures and token expiration.

Remediation:

- Continue validating JWT signatures and expiration for every protected endpoint.

Status: **PASS**

Test 4 — Authorization Testing

OWASP Category	Severity	Status
A01 – Broken Access Control	None	PASS

Objective: Verify that users cannot access resources without proper authorization.

Tools Used: Burp Repeater

Steps Performed:

- Requested protected APIs without authentication.
- Attempted to access driver resources using unauthorized requests.
- Verified authorization enforcement.

Evidence: Protected endpoints tested without a valid session.

Observed Result: Unauthorized requests were rejected. Protected resources required valid authentication.

Analysis: Authorization checks are enforced for protected endpoints.

Remediation:

- Continue validating user identity before granting access.

Status: **PASS**

Test 5 — Login Rate Limiting

OWASP Category	Severity	Status
A07 – Identification & Authentication Failures	Medium	FAIL

Objective: Verify protection against brute-force attacks.

Tools Used: Burp Intruder

Steps Performed:

- Captured login request.
- Sent multiple invalid password attempts.
- Observed server responses.
- Attempted login with valid credentials after repeated failures.

Evidence: Approximately ten consecutive failed login attempts were performed.

Observed Result: Every request returned 401 Unauthorized. No account lockout. No response delay. No CAPTCHA. Correct credentials continued to work immediately after repeated failures.

Analysis: The application does not implement brute-force protection or request throttling.

Finding: **Missing Login Rate Limiting**

Remediation:

- Implement rate limiting.
- Implement progressive delays.
- Implement account lockout.
- Add CAPTCHA after repeated failures.

Status: **FAIL**

Test 6 – SQL Injection (Authentication)

OWASP Category	Severity	Status
A03 – Injection	None	PASS

Objective: Verify whether SQL Injection vulnerabilities exist in the login functionality.

Tools Used: Burp Repeater

Steps Performed:

- Injected payloads into the email and password fields.

Evidence: Payloads used: ' OR '1'='1 , admin' -- , ' UNION SELECT NULL--

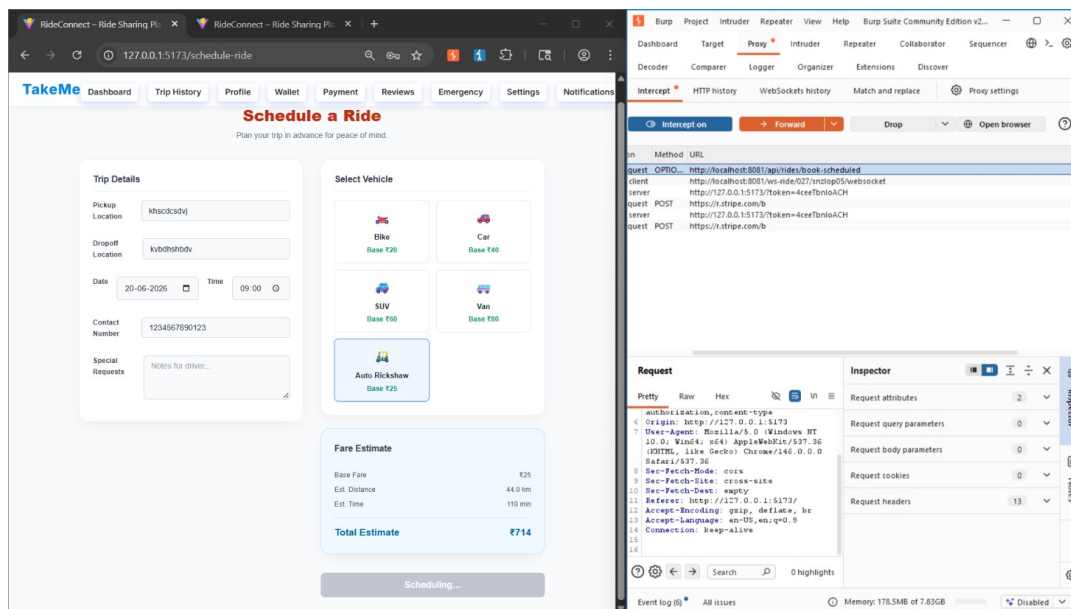


Figure — Test 6: SQL Injection (Authentication) (Burp Suite capture)

Observed Result: The application rejected all malicious payloads. Authentication bypass was unsuccessful.

Analysis: Prepared statements or parameterized queries appear to be implemented correctly.

Remediation:

- Continue using parameterized database queries.

Status: PASS

Test 7 – SQL Injection (API Parameters)

OWASP Category	Severity	Status
A03 – Injection	None	PASS

Objective: Determine whether URL parameters are vulnerable to SQL Injection.

Tools Used: Burp Repeater**Steps Performed:**

- Injected SQL payloads into API parameters.

Evidence: Examples: GET /api/profile/1' , GET /api/profile/1 OR 1=1

Observed Result: The application returned validation errors. No SQL errors or unexpected data were returned.

Analysis: No SQL Injection vulnerability was identified.

Remediation:

- Continue validating numeric parameters.

Status: **PASS**

Test 8 — JWT Claim Tampering

OWASP Category	Severity	Status
A08 – Software and Data Integrity Failures	None	PASS

Objective: Verify that modified JWT payloads are rejected.

Tools Used: Burp Decoder, Burp Repeater

Steps Performed:

- Decoded the JWT.
- Modified "role":"Driver" to "role":"Admin".
- Reused the original signature.
- Sent the modified token.

Evidence: Modified JWT payload sent with original signature.

Observed Result: The server rejected the modified JWT.

Analysis: JWT signature validation is correctly implemented.

Remediation:

- Continue enforcing signature verification.

Status: **PASS**

Test 9 — Session Management

OWASP Category	Severity	Status
A07 – Identification & Authentication Failures	None	PASS

Objective: Verify secure handling of authenticated sessions.

Tools Used: Browser, Burp Suite

Steps Performed:

- Logged in.
- Accessed authenticated endpoints.
- Removed JWT.
- Attempted access again.

Evidence: Authenticated endpoint access tested before and after JWT removal.

Observed Result: The application denied access after removing the JWT.

Analysis: Session validation is functioning correctly.

Remediation:

- Continue invalidating unauthorized sessions.

Status: **PASS**

Test 10 — Password Storage Verification

OWASP Category	Severity	Status
A02 – Cryptographic Failures	Medium	FAIL

Objective: Verify whether plaintext passwords are stored or transmitted.

Tools Used: Burp Repeater, API Response Inspection

Steps Performed:

- Retrieved user profile responses.
- Inspected returned JSON.
- Reviewed password-related fields.

Evidence: Example response field: "password":"\$2a\$10\$..."

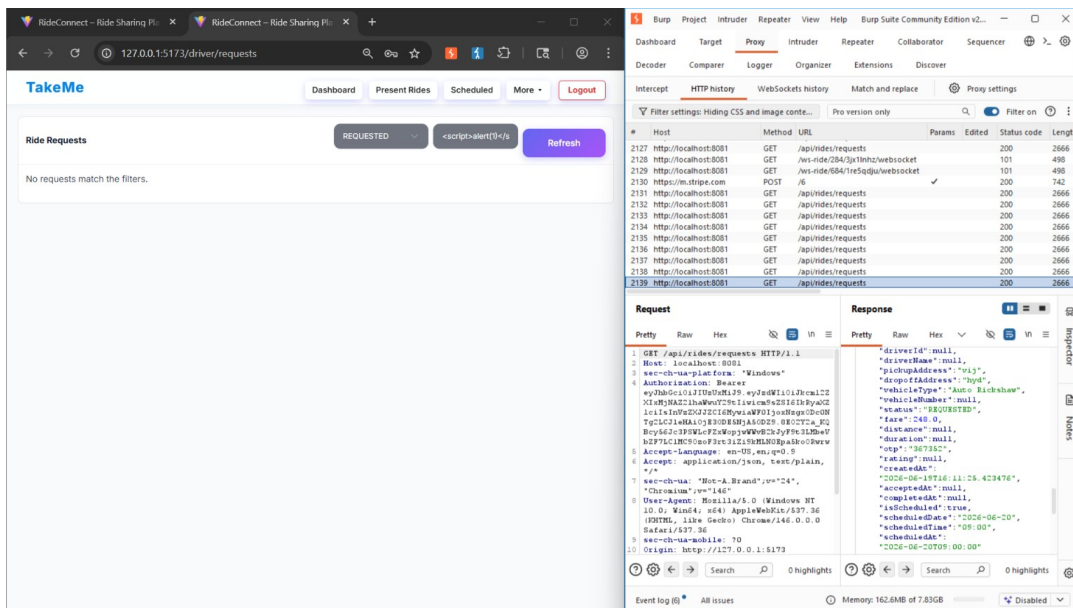


Figure — Test 10: Password Storage Verification (Burp Suite capture)

Observed Result: Passwords were stored using BCrypt hashing. However, the API returned password hashes to the client.

Analysis: Although passwords are securely hashed, exposing password hashes through API responses constitutes sensitive information disclosure.

Finding: **Sensitive Information Exposure – Password Hash Disclosure**

Remediation:

- Remove the password field from all API responses.
- Use DTOs instead of exposing entity objects.
- Return only required profile information.

Status: **FAIL**

Test 11 — Driver Profile IDOR

OWASP Category	Severity	Status
A01 – Broken Access Control	High	FAIL

Objective: Verify whether an authenticated user can access another driver's profile by modifying the profile ID.

Tools Used: Burp Suite Proxy, Burp Suite Repeater

Steps Performed:

- Logged in as Driver A.
- Captured the request GET /api/profile/3?role=Driver.
- Sent the request to Burp Repeater.
- Modified the profile ID from 3 to 4 (and other IDs).

- Sent the modified request while keeping the same JWT.

Evidence: Endpoint tested: GET /api/profile/{id}?role=Driver

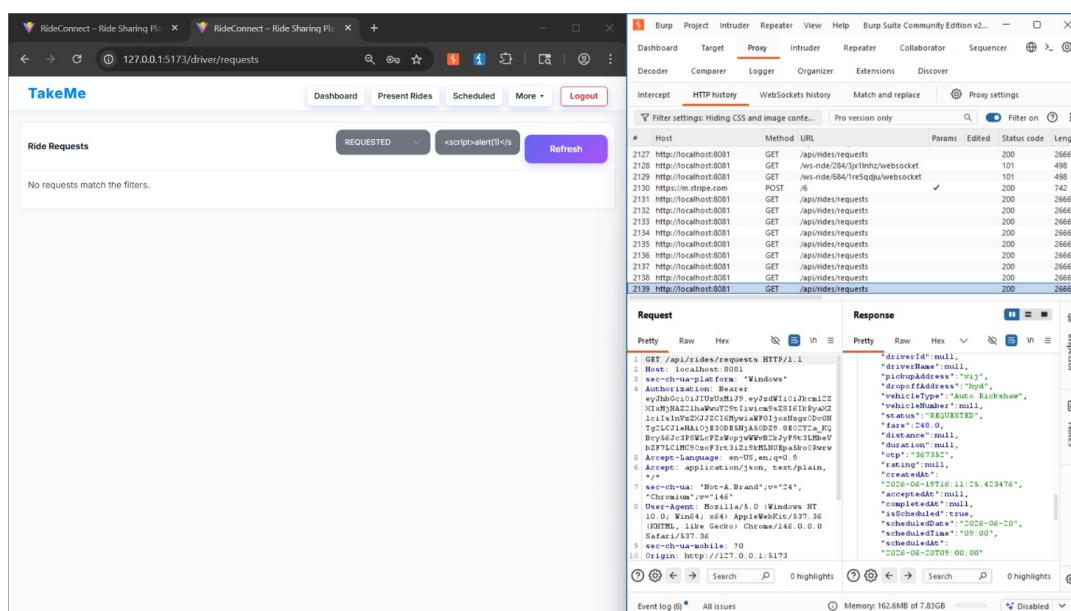


Figure — Test 11: Driver Profile IDOR (Burp Suite capture)

Observed Result: The application returned another driver's profile information without verifying resource ownership, including driver name, email, phone number, license number, and password hash.

Analysis: The application authorizes requests based only on the supplied profile ID rather than validating ownership against the authenticated JWT. This allows any authenticated driver to enumerate other driver profiles.

Finding: **Insecure Direct Object Reference (IDOR)**

Remediation:

- Never trust client-supplied IDs.
- Compare the authenticated user's ID from the JWT with the requested resource.
- Return 403 Forbidden for unauthorized access.

Status: **FAIL**

Test 12 — Password Hash Exposure

OWASP Category	Severity	Status
A02 – Cryptographic Failures	Medium	FAIL

Objective: Determine whether sensitive authentication data is exposed through profile APIs.

Tools Used: Burp Suite Repeater

Steps Performed:

- Retrieved driver profile information.

- Inspected the JSON response.

Evidence: Example: { "password": "\$2a\$10\$..." }

Observed Result: The response contained a BCrypt password hash.

Analysis: Although BCrypt is a secure hashing algorithm, password hashes should never be returned to clients. This increases the attack surface for offline password cracking.

Finding: Sensitive Information Disclosure

Remediation:

- Use DTOs and exclude the password field from all API responses.

Status: FAIL

Test 13 — Driver Scheduled Ride IDOR

OWASP Category	Severity	Status
A01 – Broken Access Control	High	FAIL

Objective: Verify whether drivers can access ride information belonging to other drivers.

Tools Used: Burp Repeater

Steps Performed:

- Captured a scheduled ride request.
- Modified the ride/driver identifier.
- Replayed the request.

Evidence: Endpoint tested: GET /api/rides/driver/{id}/scheduled

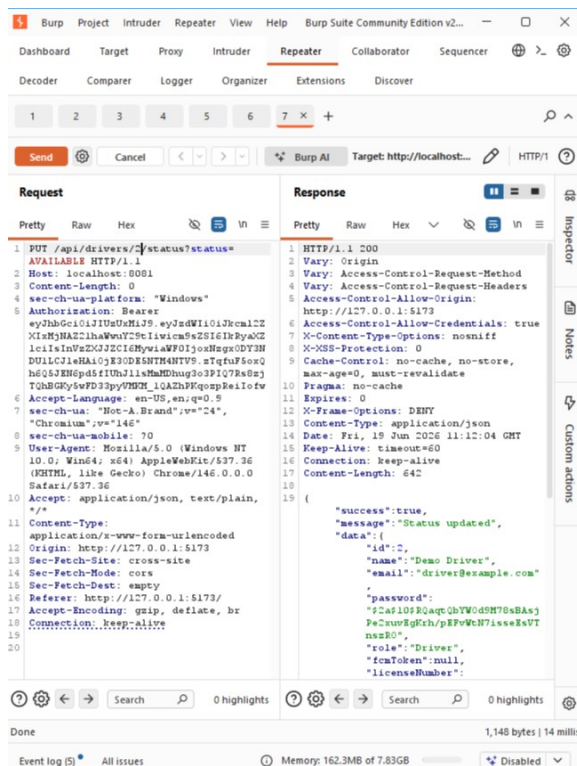


Figure — Test 13: Driver Scheduled Ride IDOR (Burp Suite capture)

Observed Result: The application returned ride details belonging to another driver.

Analysis: The backend failed to verify ownership of the ride before returning data.

Finding: Broken Access Control (IDOR) – Driver Scheduled Ride Disclosure

Remediation:

- Validate ride ownership before returning ride details.

Status: **FAIL**

Test 14 — Driver Status Manipulation

OWASP Category	Severity	Status
A01 – Broken Access Control	High	FAIL

Objective: Verify whether a driver can manipulate another driver's status.

Tools Used: Burp Repeater

Steps Performed:

- Intercepted the status update request.
- Modified the driver ID.
- Sent the modified request.

Evidence: Endpoint tested: PUT /api/drivers/{id}/status

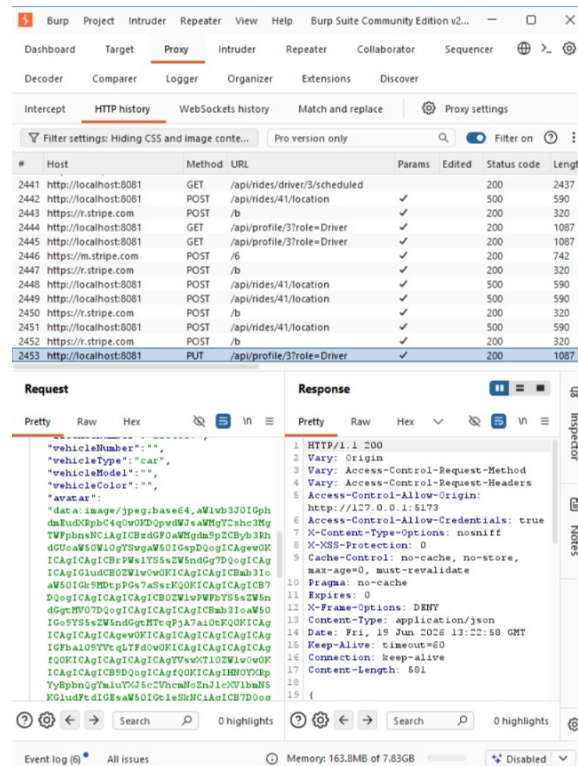


Figure — Test 14: Driver Status Manipulation (Burp Suite capture)

Observed Result: Another driver's status was successfully modified.

Analysis: The backend trusts the client-supplied identifier without verifying ownership.

Finding: **Broken Access Control**

Remediation:

- Authorize every update using the authenticated user's identity rather than request parameters.

Status: **FAIL**

Test 15 — HTML File Upload

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether the application accepts HTML files as profile pictures.

Tools Used: Burp Proxy, Browser

Steps Performed:

- Selected an HTML file.
- Uploaded it as a profile picture.

Evidence: HTML file uploaded as profile picture via the avatar upload feature.

Observed Result: The upload was rejected.

Analysis: The application correctly validates file types.

Remediation:

- Continue validating MIME type and file extension.

Status: **PASS**

Test 16 — SVG Upload Validation

OWASP Category	Severity	Status
A03 – Injection	None	PASS

Objective: Verify whether SVG files containing embedded scripts can be uploaded and rendered.

Tools Used: Browser, Burp Suite

Steps Performed:

- Uploaded an SVG image containing XML content with an embedded script payload.
- Viewed the uploaded profile picture.

Evidence: SVG payload containing embedded script content uploaded as a profile avatar.

Observed Result: The file uploaded successfully. The application displayed it as an image. No JavaScript execution occurred.

Analysis: The application allows SVG uploads but safely renders them as images. No Stored XSS was observed.

Remediation:

- Continue sanitizing SVG files and consider rasterizing uploads for additional protection.

Status: **PASS**

Test 17 — Plain Text File Upload

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether unsupported file types are accepted.

Tools Used: Browser

Steps Performed:

- Uploaded a plain text file as the profile image.

Evidence: Plain .txt file submitted via the avatar upload feature.

Observed Result: The upload was rejected.

Analysis: The application correctly prevents unsupported file uploads.

Remediation:

- Maintain strict allow-list validation.

Status: **PASS**

Test 18 — File Content Validation

OWASP Category	Severity	Status
A05 – Security Misconfiguration	Medium	FAIL

Objective: Verify whether uploaded files are validated beyond their file extension.

Tools Used: Burp Repeater

Steps Performed:

- Modified uploaded file metadata and attempted to bypass validation.

Evidence: Manipulated file metadata sent via Repeater during avatar upload.

Observed Result: The application accepted files based primarily on file type, with limited content inspection.

Analysis: Although no code execution occurred, deeper validation of uploaded content is recommended.

Finding: **Weak File Content Validation**

Remediation:

- Validate MIME type.
- Verify file signatures (magic bytes).
- Sanitize SVG and image metadata.
- Scan uploads before storage.

Status: **FAIL**

Test 19 — Stored Cross-Site Scripting (XSS)

OWASP Category	Severity	Status
A03 – Injection	None	PASS

Objective: Verify whether user-supplied input is sanitized before being stored and rendered.

Tools Used: Burp Repeater

Steps Performed:

- Injected a JavaScript payload into a profile field.
- Saved the profile and revisited pages displaying the stored value.

Evidence: Payload used: `<img src=x onerror=alert(1)>`

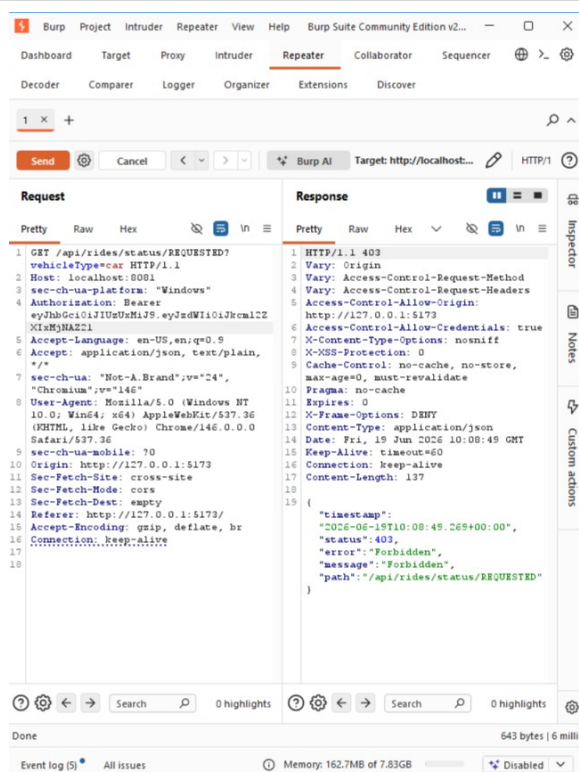


Figure — Test 19: Stored Cross-Site Scripting (XSS) (Burp Suite capture)

Observed Result: The payload was stored in the application. No JavaScript execution occurred because the application rendered it safely.

Analysis: The application prevented Stored XSS by safely handling user input on the rendering paths tested.

Remediation:

- Continue output encoding and input validation.

Status: **PASS**

Test 20 — Reflected Cross-Site Scripting (XSS)

OWASP Category	Severity	Status
A03 - Injection	None	PASS

Objective: Verify whether reflected user input is executed by the browser.

Tools Used: Burp Repeater

Steps Performed:

- Injected common XSS payloads into request parameters.

Evidence: Payloads used: `<script>alert(1)</script>` , `<img src=x onerror=alert(1)>`

Observed Result: The application rejected or safely encoded the supplied payloads. No JavaScript execution occurred.

Analysis: No reflected XSS vulnerability was identified.

Remediation:

- Continue output encoding and implement a Content Security Policy (CSP) as an additional defense.

Status: **PASS**

Test 21 — Rider Identity Tampering (IDOR)

OWASP Category	Severity	Status
A01 – Broken Access Control	Critical	FAIL

Objective: Verify whether a user can modify the rider identifier in ride-related requests to impersonate another rider.

Tools Used: Burp Suite Proxy, Burp Suite Repeater

Steps Performed:

- Logged in as a Rider.
- Intercepted the ride booking request.
- Modified the userId/riderId parameter.
- Replayed the request with the same JWT.

Evidence: Endpoint: POST /api/rides/book . Modified parameter: "userId": 5 → "userId": 2

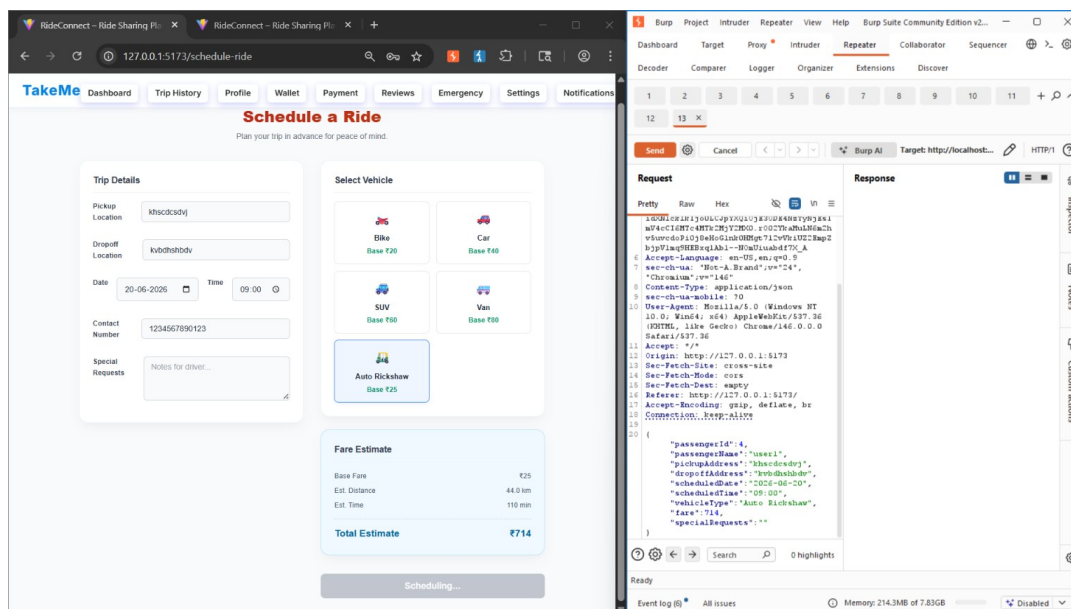


Figure — Test 21: Rider Identity Tampering (IDOR) (Burp Suite capture)

Observed Result: The backend accepted the modified rider ID and processed the request.

Analysis: The application trusted the client-supplied rider identifier instead of using the authenticated user's identity from the JWT. This allows a malicious user to perform actions on behalf of another rider.

Finding: **Broken Access Control – Insecure Direct Object Reference (IDOR)**

Remediation:

- Ignore client-supplied user IDs.
- Always derive the user identity from the authenticated JWT.
- Validate ownership before processing requests.

Status: FAIL**Test 22 — Unauthorized Ride Acceptance**

OWASP Category	Severity	Status
A01 – Broken Access Control	Critical	FAIL

Objective: Verify whether a driver can accept rides that do not belong to them.**Tools Used:** Burp Suite Repeater**Steps Performed:**

- Captured the ride acceptance request.
- Modified the ride ID.
- Replayed the request.

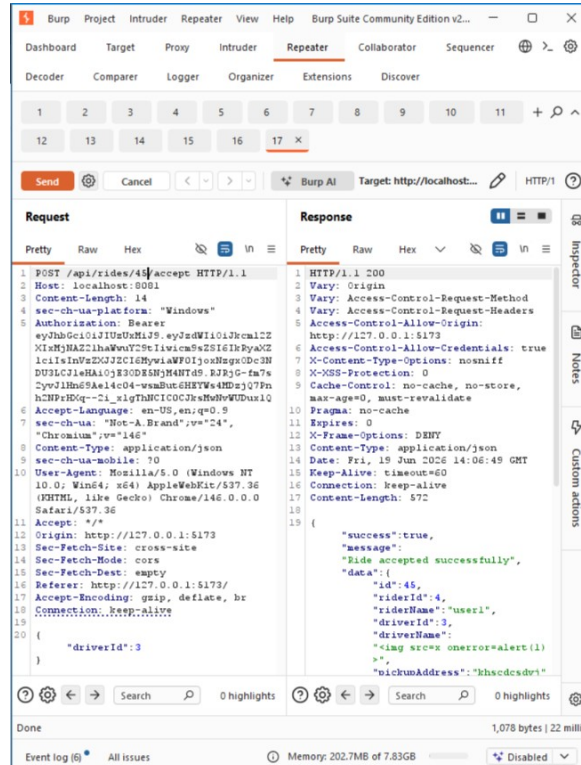
Evidence: Endpoint: POST /api/rides/25/accept → POST /api/rides/18/accept

Figure — Test 22: Unauthorized Ride Acceptance (Burp Suite capture)

Observed Result: The backend accepted the modified ride ID.

Analysis: The application failed to verify ownership before allowing ride acceptance.

Finding: **Broken Access Control (IDOR) – Unauthorized Ride Acceptance**

Remediation:

- Validate that the authenticated driver owns the ride before processing acceptance.

Status: **FAIL**

Test 23 — OTP Enforcement

OWASP Category	Severity	Status
A07 – Identification & Authentication Failures	None	PASS

Objective: Verify whether rides can be started without successful OTP verification.

Tools Used: Burp Suite, Browser

Steps Performed:

- Accepted a ride.
- Attempted to start the ride without entering the OTP.
- Observed the response.

Evidence: Ride-start flow tested without supplying the rider's OTP.

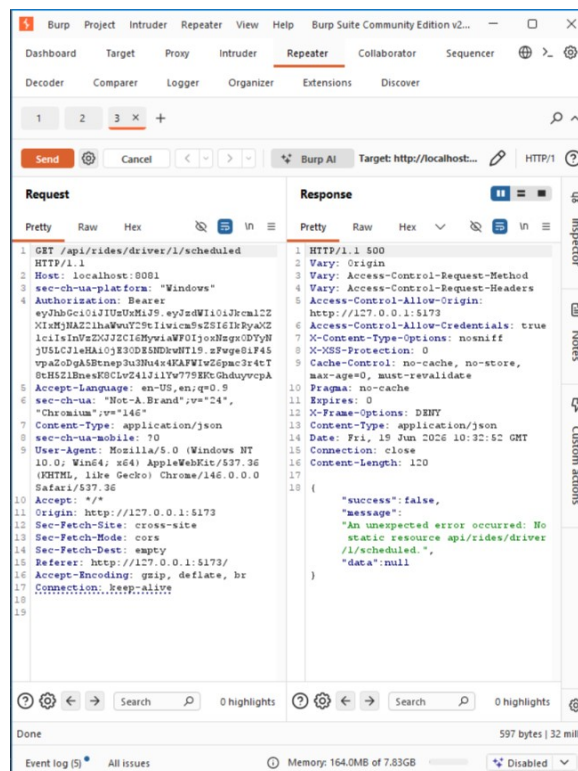


Figure — Test 23: OTP Enforcement (Burp Suite capture)

Observed Result: The application prevented the ride from starting until the correct OTP was verified.

Analysis: OTP verification is correctly enforced.

Remediation:

- Continue validating OTP server-side.

Status: **PASS**

Test 24 — Fare Manipulation

OWASP Category	Severity	Status
A04 – Insecure Design	High	FAIL

Objective: Determine whether the client can manipulate ride fare values.

Tools Used: Burp Repeater

Steps Performed:

- Captured the booking request.
- Modified the fare value.
- Sent the modified request.

Evidence: Modified parameter: "fare":250 → "fare":1 (also tested with inflated values)

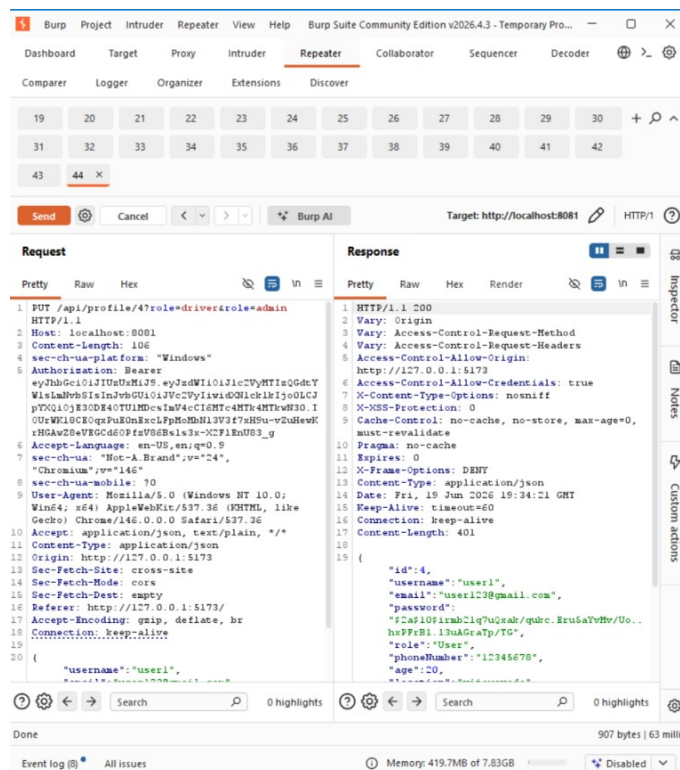


Figure — Test 24: Fare Manipulation (Burp Suite capture)

Observed Result: The backend accepted the manipulated fare.

Analysis: Fare calculation relies on client-supplied data rather than secure server-side calculations.

Finding: **Business Logic Vulnerability**

Remediation:

- Calculate fares exclusively on the server.
- Ignore client-supplied pricing values.
- Validate all financial transactions.

Status: **FAIL**

Test 25 — Intruder Brute Force Testing

OWASP Category	Severity	Status
A07 – Identification & Authentication Failures	Medium	FAIL

Objective: Verify the application's resistance to repeated authentication attempts.

Tools Used: Burp Intruder

Steps Performed:

- Captured the login request.
- Configured password payloads.
- Sent multiple authentication attempts.

Evidence: Automated Intruder attack against the login endpoint with a credential wordlist.

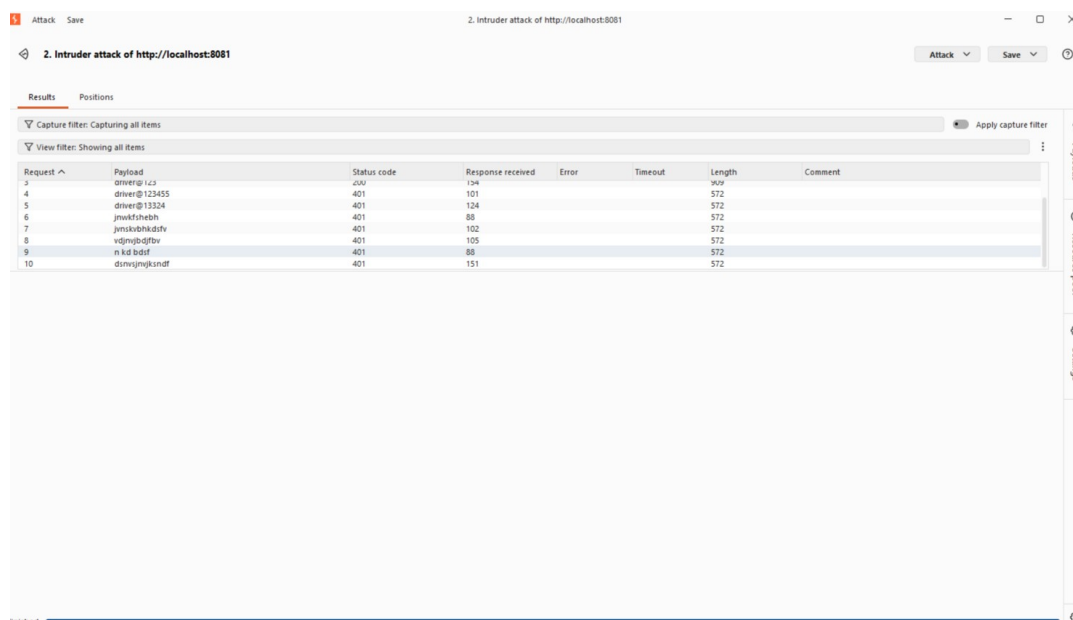


Figure — Test 25: Intruder Brute Force Testing (Burp Suite capture)

Observed Result: Repeated authentication attempts were allowed without throttling or account lockout.

Analysis: The application lacks brute-force protection.

Finding: **Missing Rate Limiting**

Remediation:

- Implement account logout.
- Introduce rate limiting.
- Apply CAPTCHA after repeated failures.

Status: **FAIL**

Test 26 — JWT Structure Analysis

OWASP Category	Severity	Status
A08 – Software and Data Integrity Failures	None	PASS

Objective: Verify that JWT tokens contain the expected security claims.

Tools Used: Burp Decoder

Steps Performed:

- Decoded the JWT.
- Verified algorithm, subject, expiration, issued-at, user ID, and role claims.

Evidence: JWT decoded via Burp Decoder.

Observed Result: JWT contained: HS512 algorithm, sub, role, userId, iat, exp claims.

Analysis: The JWT follows secure construction practices.

Remediation:

- Continue using strong signing algorithms and expiration validation.

Status: **PASS**

Test 27 — JWT Signature Validation

OWASP Category	Severity	Status
A08 – Software and Data Integrity Failures	None	PASS

Objective: Verify whether modified JWT payloads are accepted.

Tools Used: Burp Decoder, Burp Repeater

Steps Performed:

- Modified "role":"Driver" to "role":"Admin" without recalculating the signature.

Evidence: Tampered JWT payload sent with the original signature intact.

Observed Result: The backend rejected the modified token.

Analysis: JWT signatures are properly verified.

Remediation:

- Maintain strict signature verification.

Status: **PASS**

Test 28 — Expired JWT Validation

OWASP Category	Severity	Status
A07 – Identification & Authentication Failures	None	PASS

Objective: Verify whether expired JWTs are rejected.

Tools Used: Burp Repeater

Steps Performed:

- Submitted an expired or invalid JWT.

Evidence: Expired JWT submitted to a protected endpoint.

Observed Result: Authentication failed.

Analysis: Token expiration is correctly enforced.

Remediation:

- Continue validating expiration timestamps.

Status: **PASS**

Test 29 — Missing JWT Validation

OWASP Category	Severity	Status
A01 – Broken Access Control	None	PASS

Objective: Verify access control when authentication tokens are omitted.

Tools Used: Burp Repeater

Steps Performed:

- Removed the Authorization header.

Evidence: Protected endpoint requested without any Authorization header.

Observed Result: Protected resources returned 401 Unauthorized.

Analysis: Authentication is correctly enforced.

Remediation:

- Continue rejecting unauthenticated requests.

Status: **PASS**

Test 30 — Malformed JWT Validation

OWASP Category	Severity	Status
A08 – Software and Data Integrity Failures	None	PASS

Objective: Verify server behavior when malformed JWTs are supplied.

Tools Used: Burp Repeater

Steps Performed:

- Replaced the Authorization header with an invalid JWT (e.g. Authorization: Bearer invalid.token.value).

Evidence: Malformed token submitted to a protected endpoint.

Observed Result: The server rejected the malformed token. No internal server errors were observed.

Analysis: JWT validation is correctly implemented.

Remediation:

- Continue validating token format before processing requests.

Status: **PASS**

Test 31 — Mass Assignment

OWASP Category	Severity	Status
A01 – Broken Access Control	High	FAIL

Objective: Verify whether sensitive object properties can be modified by adding additional JSON fields to update requests.

Tools Used: Burp Suite Repeater

Steps Performed:

- Logged in as a normal user.
- Intercepted the profile update request.
- Added unauthorized fields to the JSON body.

- Sent the request.
- Retrieved the updated profile.

Evidence: Endpoint: PUT /api/profile/{id}. Modified body included: "walletBalance":999999, "verified":true, "rating":5, "active":true

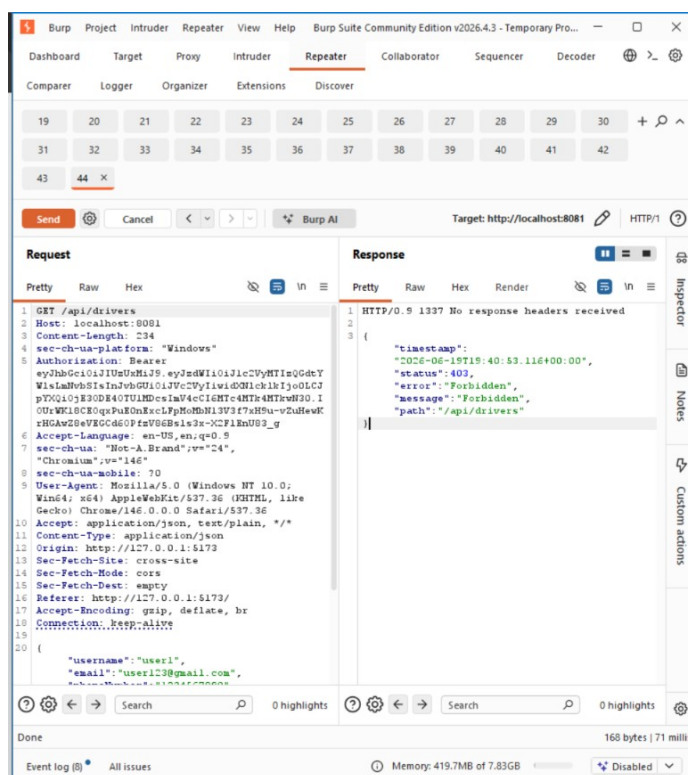


Figure — Test 31: Mass Assignment (Burp Suite capture)

Observed Result: The backend accepted and updated protected fields that should only be controlled by server-side business logic.

Analysis: The application binds incoming JSON directly to backend objects without restricting which properties may be updated. This allows attackers to modify security-sensitive attributes.

Finding: Mass Assignment Vulnerability

Remediation:

- Use DTOs instead of JPA entities.
- Whitelist editable fields.
- Ignore sensitive fields such as `walletBalance`, `verified`, `rating`, `active`, `totalRides`, `totalEarnings`.

Status: **FAIL**

Test 32 — Privilege Escalation

OWASP Category	Severity	Status
A01 – Broken Access Control	None	PASS

Objective: Determine whether a normal user can elevate privileges by modifying the role field.

Tools Used: Burp Repeater

Steps Performed:

- Modified "role":"Driver" to "role":"Admin".
- Sent the modified request.

Evidence: Role field tampered with directly in the profile update request body.

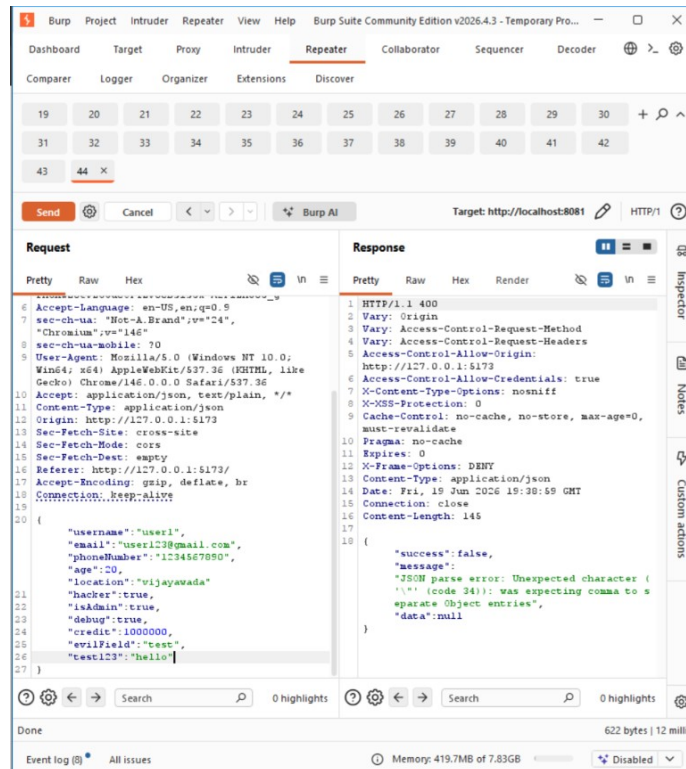


Figure — Test 32: Privilege Escalation (Burp Suite capture)

Observed Result: The application ignored the modified role. The account remained assigned to its original role.

Analysis: Role-based authorization is correctly enforced.

Remediation:

- Continue validating roles on the server side.

Status: **PASS**

Test 33 — HTTP Method Tampering

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify that endpoints only accept intended HTTP methods.

Tools Used: Burp Repeater

Steps Performed:

- Changed GET to POST, PUT, PATCH, DELETE on various endpoints.

Evidence: Endpoint methods substituted and replayed via Repeater.

Observed Result: The server rejected unsupported methods. No unauthorized actions were performed.

Analysis: Spring Boot correctly restricts HTTP methods.

Remediation:

- Continue defining explicit HTTP methods for each controller endpoint.

Status: **PASS**

Test 34 — HTTP Parameter Pollution

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether duplicate query parameters influence authorization or application logic.

Tools Used: Burp Repeater

Steps Performed:

- Modified GET /api/profile/3?role=Driver to include duplicate role parameters in both orders.

Evidence: GET /api/profile/3?role=Driver&role=Admin and GET /api/profile/3?role=Admin&role=Driver

Observed Result: The backend processed the authenticated user's request correctly. Duplicate role parameters did not result in privilege escalation or unauthorized data access.

Analysis: The application safely handled duplicate parameters.

Remediation:

- Continue validating authorization using authenticated user context rather than query parameters.

Status: **PASS**

Test 35 — Error Handling & Information Disclosure

OWASP Category	Severity	Status
A05 – Security Misconfiguration	Informational	PASS

Objective: Verify whether application errors expose internal implementation details.

Tools Used: Burp Repeater

Steps Performed:

- Requested invalid resources.

- Submitted malformed requests.
- Observed error responses.

Evidence: Various invalid/malformed requests sent to backend API endpoints.

Observed Result: The application returned generic JSON error messages. No SQL queries, Java stack traces, or filesystem paths were disclosed. However, some invalid requests returned the default Tomcat error page.

Analysis: Error handling is generally secure.

Remediation:

- Implement custom Spring Boot exception handlers using @ControllerAdvice and a custom ErrorController.

Status: **PASS**

Test 36 — Unexpected JSON Fields

OWASP Category	Severity	Status
A04 – Insecure Design	None	PASS

Objective: Determine whether unknown JSON properties are processed by the backend.

Tools Used: Burp Repeater

Steps Performed:

- Added unexpected fields to a request body: "hacker":true, "isAdmin":true, "credit":1000000, "evilField":"test".

Evidence: Request body with arbitrary undeclared fields sent to a tested endpoint.

Observed Result: The application ignored all unknown fields. No additional properties appeared in responses.

Analysis: On this endpoint, the backend safely ignored undeclared JSON properties.

Remediation:

- Continue validating incoming request schemas.

Status: **PASS**

Test 37 — Forced Browsing

OWASP Category	Severity	Status
A01 – Broken Access Control	None	PASS

Objective: Verify whether hidden or administrative endpoints are accessible.

Tools Used: Burp Repeater

Steps Performed:

- Requested multiple administrative endpoints.

Evidence: Examples: GET /api/admin , GET /api/drivers , GET /actuator , GET /swagger-ui , GET /v3/api-docs

Observed Result: Responses included 401 Unauthorized, 403 Forbidden, and 404 Not Found. No administrative information was exposed.

Analysis: Access controls correctly protect administrative resources.

Remediation:

- Disable unused endpoints in production.

Status: **PASS**

Test 38 — CORS Validation

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether unauthorized origins are permitted.

Tools Used: Burp Repeater

Steps Performed:

- Modified the Origin header to http://evil.com.

Evidence: Cross-origin request sent with an untrusted Origin header.

Observed Result: The response did not include Access-Control-Allow-Origin for the malicious origin.

Analysis: The application correctly rejects unauthorized cross-origin requests.

Remediation:

- Maintain an explicit allow-list of trusted origins.

Status: **PASS**

Test 39 — Clickjacking Protection

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify protection against clickjacking attacks.

Tools Used: Burp Repeater

Steps Performed:

- Inspected HTTP response headers.

Evidence: Response headers reviewed for frame-protection directives.

Observed Result: Observed X-Frame-Options: DENY.

Analysis: The application cannot be embedded within iframes.

Remediation:

- Maintain X-Frame-Options: DENY or Content-Security-Policy: frame-ancestors 'none'.

Status: **PASS**

Test 40 — Content Security Policy (CSP)

OWASP Category	Severity	Status
A05 – Security Misconfiguration	Informational	INFORMATIONAL

Objective: Verify whether the application implements a Content Security Policy.

Tools Used: Burp Suite

Steps Performed:

- Inspected response headers.

Evidence: HTTP response headers reviewed across multiple endpoints.

Observed Result: No Content-Security-Policy header was present.

Analysis: Although no XSS vulnerability was identified, the absence of CSP reduces browser-side defense against future client-side injection attacks.

Finding: **Missing Content Security Policy**

Remediation:

- Configure Spring Security to return a restrictive CSP header, e.g. default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none';

Status: **INFORMATIONAL**

Test 41 — Cache Control Validation

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether authenticated responses are protected from browser and proxy caching.

Tools Used: Burp Suite Repeater, HTTP History

Steps Performed:

- Logged into the application.
- Accessed authenticated endpoints.

- Inspected HTTP response headers.

Evidence: Headers observed: Cache-Control: no-cache, no-store, max-age=0, must-revalidate ; Pragma: no-cache ; Expires: 0

Observed Result: Authenticated responses were configured with appropriate cache prevention headers.

Analysis: Sensitive information cannot be stored in browser cache or intermediate proxies, reducing the risk of data exposure on shared systems.

Remediation:

- Maintain current cache-control configuration for all authenticated endpoints.

Status: **PASS**

Test 42 — HTTP OPTIONS Method Validation

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether unnecessary HTTP methods are enabled.

Tools Used: Burp Suite Repeater

Steps Performed:

- Sent an HTTP OPTIONS request to API endpoints, e.g. OPTIONS /api/profile/3.

Evidence: Response: Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS, PATCH

Observed Result: Only standard REST methods were enabled. TRACE, TRACK, and CONNECT methods were not advertised.

Analysis: The API exposes only expected HTTP methods required by the application.

Remediation:

- Continue disabling unnecessary HTTP methods.

Status: **PASS**

Test 43 — HTTP TRACE Method

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether the HTTP TRACE method is enabled.

Tools Used: Burp Suite Repeater

Steps Performed:

- Sent TRACE / HTTP/1.1 to the backend host.

Evidence: Server Response: HTTP/1.1 400 Bad Request

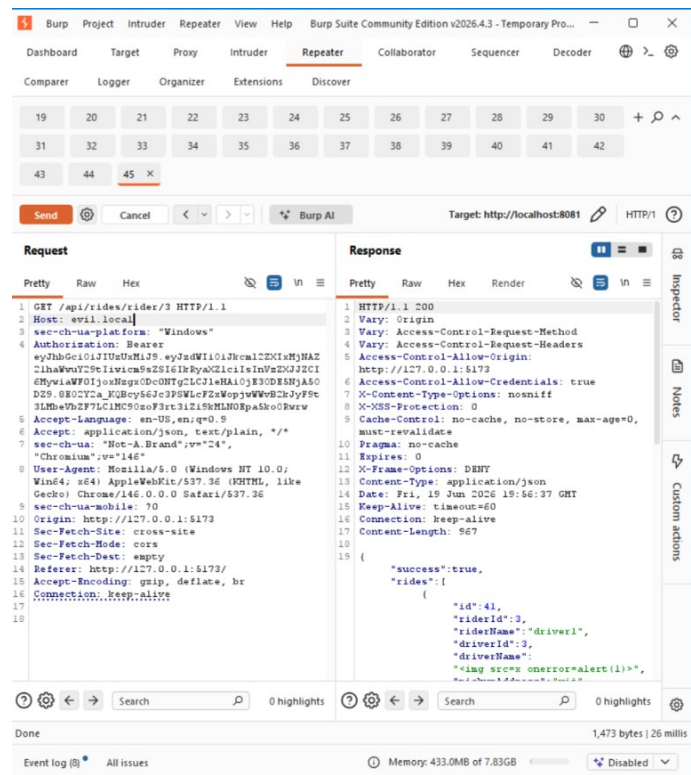


Figure — Test 43: HTTP TRACE Method (Burp Suite capture)

Observed Result: TRACE requests were rejected.

Analysis: The application is not vulnerable to Cross-Site Tracing (XST).

Remediation:

- Maintain TRACE disabled in production.

Status: PASS

Test 44 — Directory Traversal

OWASP Category	Severity	Status
A05 – Security Misconfiguration	Low	INFORMATIONAL

Objective: Verify whether attackers can access sensitive files using directory traversal payloads.

Tools Used: Burp Suite Repeater

Steps Performed:

- Attempted payloads against file/path-accepting parameters.

Evidence: Payloads: ../../../../etc/passwd , ../../../../%2f..%2f..%2f..%2fetc/passwd , ../../../../Windows/win.ini

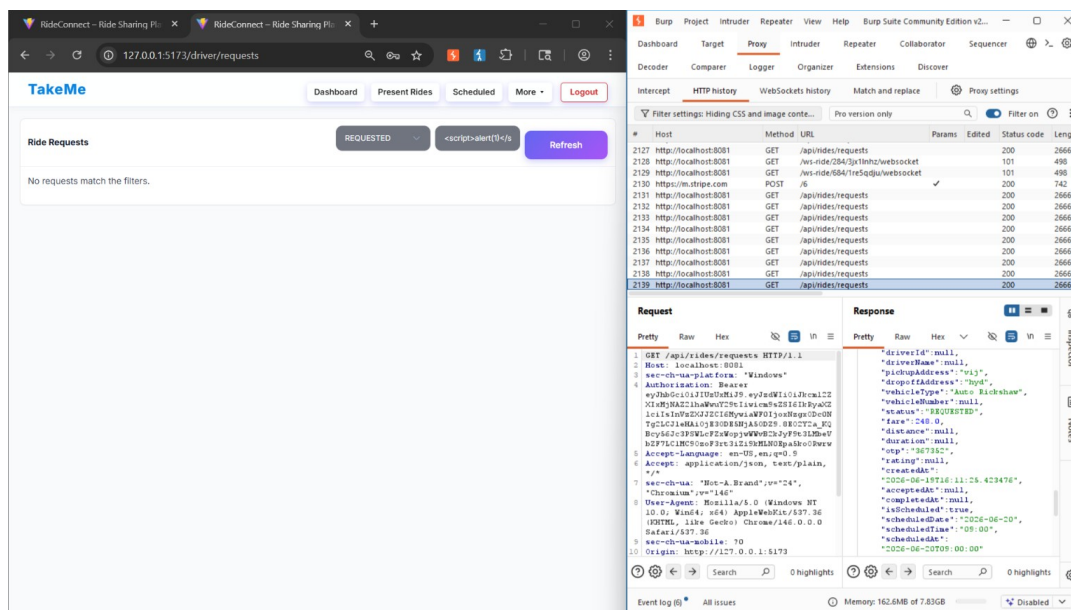


Figure — Test 44: Directory Traversal (Burp Suite capture)

Observed Result: The server rejected all traversal attempts. However, the response included a Java exception name and Apache Tomcat parser information.

Analysis: Directory traversal was successfully prevented. However, the error response reveals internal implementation details that could aid reconnaissance.

Finding: Information Disclosure through Verbose Error Messages

Remediation:

- Configure Spring Boot to return generic error messages and disable detailed server error responses in production.

Status: INFORMATIONAL

Test 45 — Host Header Injection

OWASP Category	Severity	Status
A05 – Security Misconfiguration	None	PASS

Objective: Verify whether the application trusts the client-supplied Host header.

Tools Used: Burp Suite Repeater

Steps Performed:

- Modified Host: localhost:8081 to Host: evil.local.

Evidence: Request replayed with a spoofed Host header against an authenticated endpoint.

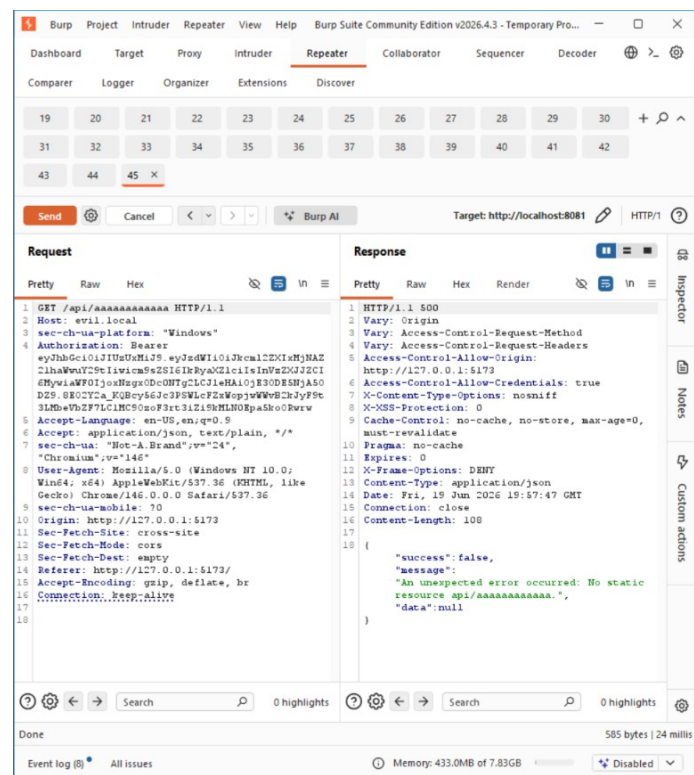


Figure — Test 45: Host Header Injection (Burp Suite capture)

Observed Result: The application ignored the modified Host header. No redirects or generated URLs used the malicious host.

Analysis: Host header validation is correctly implemented for the flows tested.

Remediation:

- Continue validating trusted host names.

Status: **PASS**

Test 46 — Security Header Review

OWASP Category	Severity	Status
A05 – Security Misconfiguration	Low	INFORMATIONAL

Objective: Review important HTTP security headers.

Tools Used: Burp Suite Repeater

Steps Performed:

- Inspected response headers from authenticated endpoints.

Evidence: Observed: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Cache-Control: no-store. Not observed: Content-Security-Policy.

Observed Result: Most recommended security headers were configured correctly. Content Security Policy was absent.

Analysis: The application follows several HTTP security best practices. The absence of CSP reduces browser-side protection against client-side attacks.

Finding: **Missing Content Security Policy**

Remediation:

- Configure a restrictive CSP policy using Spring Security.

Status: **INFORMATIONAL**

Test 47 — Error Handling Validation

OWASP Category	Severity	Status
A05 – Security Misconfiguration	Low	INFORMATIONAL

Objective: Verify whether unexpected requests expose sensitive application information.

Tools Used: Burp Suite Repeater

Steps Performed:

- Requested invalid endpoints, e.g. GET /api/aaaaaaaaaaaaaaaaaaaaaa.

Evidence: Response: { "success":false, "message":"An unexpected error occurred." }

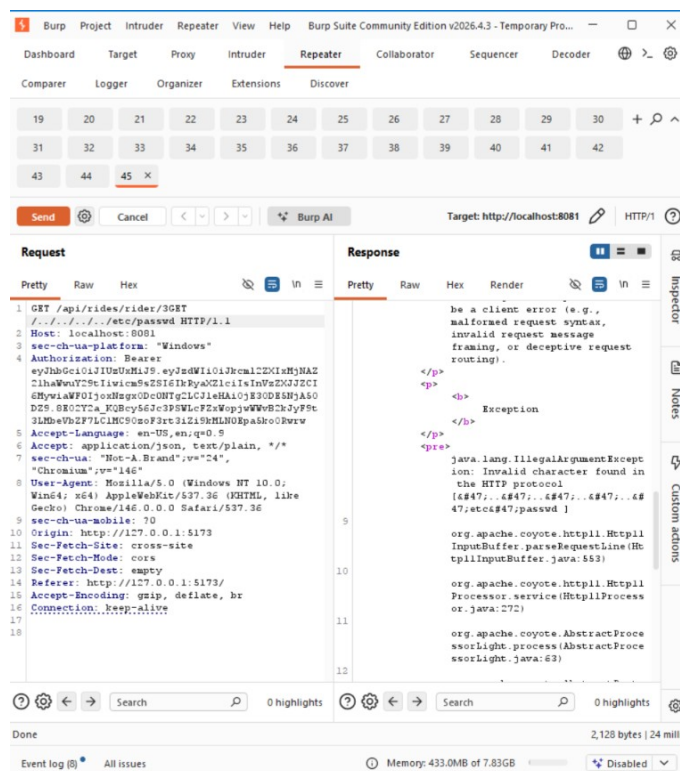


Figure — Test 47: Error Handling Validation (Burp Suite capture)

Observed Result: No Java stack traces, SQL queries, or filesystem paths were exposed. However, invalid resources generated HTTP 500 responses instead of HTTP 404 responses.

Analysis: The application handles unexpected requests securely, though status-code semantics could be improved.

Finding: **Improper HTTP Status Code Handling**

Remediation:

- Return HTTP 404 for unknown resources and reserve HTTP 500 for genuine server-side exceptions.

Status: **INFORMATIONAL**

Test 48 — Sensitive Information Disclosure

OWASP Category	Severity	Status
A02 – Cryptographic Failures	Medium	FAIL

Objective: Verify whether sensitive information is exposed through API responses.

Tools Used: Burp Suite Repeater

Steps Performed:

- Retrieved profile information.
- Inspected JSON responses.
- Reviewed returned attributes.

Evidence: Example: { "id":3, "email":"driver123@gmail.com",
"password":"\$2a\$10\$xx" }

Observed Result: The application exposed BCrypt password hashes in API responses. Additional user information such as email addresses and phone numbers was also returned.

Analysis: Although passwords are securely hashed, password hashes should never be exposed to clients. This increases the risk of offline password cracking and violates secure API design principles.

Finding: **Sensitive Information Disclosure – Password Hash Exposure**

Remediation:

- Remove password fields from all API responses.
- Use dedicated DTOs for serialization.
- Return only the minimum required user information.

Status: **FAIL**

11. OWASP Top 10 (2021) Mapping

The security assessment findings were mapped against the OWASP Top 10 (2021) categories to identify the primary security risks affecting the RideConnect web application.

OWASP Category	Vulnerability Identified	Severity	Status
A01 – Broken Access Control	Driver Profile IDOR	High	Identified
A01 – Broken Access Control	Driver Scheduled Ride IDOR	High	Identified
A01 – Broken Access Control	Driver Status Manipulation	High	Identified
A01 – Broken Access Control	Rider Identity Tampering	Critical	Identified
A01 – Broken Access Control	Unauthorized Ride Acceptance	Critical	Identified
A01 – Broken Access Control	Mass Assignment	High	Identified
A02 – Cryptographic Failures	Password Hash Exposure in API Responses	Medium	Identified
A03 – Injection	SQL Injection	None	Secure
A03 – Injection	Stored Cross-Site Scripting (XSS)	None	Secure
A03 – Injection	Reflected Cross-Site Scripting (XSS)	None	Secure
A04 – Insecure Design	Client-Side Fare Manipulation	High	Identified
A05 – Security Misconfiguration	Missing Content Security Policy	Informational	Improvement Recommended
A05 – Security Misconfiguration	Verbose Error Messages	Low	Improvement Recommended
A05 – Security Misconfiguration	Weak File Content Validation	Medium	Identified
A07 – Identification & Authentication Failures	Missing Login Rate Limiting	Medium	Identified
A07 – Identification & Authentication Failures	JWT Signature Validation	None	Secure
A07 – Identification & Authentication Failures	OTP Verification	None	Secure
A08 – Software & Data Integrity Failures	JWT Tampering Protection	None	Secure
A09 – Security Logging & Monitoring Failures	Not Assessed	N/A	Outside Scope
A10 – Server-Side Request Forgery (SSRF)	Not Applicable	N/A	Not Tested

Summary

The assessment identified that the majority of confirmed vulnerabilities fall under OWASP A01: Broken Access Control, indicating that authorization checks should be strengthened throughout the application. Business logic flaws and insecure direct object references (IDOR) represented the highest-risk findings.

Authentication controls, JWT validation, SQL Injection defenses, XSS protections, and HTTP security headers were generally implemented effectively. Additional improvements in secure configuration, API response filtering, and server-side validation would significantly improve the application's overall security posture.

12. Secure Coding Recommendations

Based on the findings identified during the penetration testing assessment, the following security improvements are recommended to strengthen the overall security posture of the RideConnect web application.

1. Strengthen Access Control

Issue: Multiple Insecure Direct Object Reference (IDOR) vulnerabilities allowed users to access or modify resources belonging to other users.

Recommendation:

- Never trust resource identifiers received from the client.
- Always derive the authenticated user's identity from the JWT.
- Verify ownership before accessing or modifying resources.
- Return 403 Forbidden for unauthorized requests.

2. Prevent Mass Assignment

Issue: Sensitive properties could be modified through JSON request bodies.

Recommendation:

- Use dedicated Data Transfer Objects (DTOs).
- Never bind request bodies directly to JPA entities.
- Whitelist editable properties.
- Ignore security-sensitive fields received from clients.

3. Protect Sensitive Information

Issue: Password hashes were returned in API responses.

Recommendation:

- Remove password fields from all API responses.
- Use DTOs for serialization.
- Return only required user information.

4. Implement Login Rate Limiting

Issue: Unlimited authentication attempts increase the risk of brute-force attacks.

Recommendation:

- Limit repeated login attempts.
- Introduce temporary account lockout.
- Implement progressive delays.
- Consider CAPTCHA after repeated failures.

5. Perform Server-Side Business Logic Validation

Issue: Ride fare values were accepted from client requests.

Recommendation:

- Calculate all fares on the server.

- Ignore client-supplied pricing values.
- Validate business rules before processing transactions.

6. Improve File Upload Validation

Issue: Uploaded files relied primarily on file type validation.

Recommendation:

- Validate file signatures (magic bytes).
- Restrict supported file types.
- Sanitize SVG files.
- Scan uploaded files before storage.
- Store uploads outside the application directory.

7. Enhance HTTP Security Headers

Issue: Content Security Policy (CSP) was not implemented.

Recommendation:

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options
- Referrer-Policy
- Permissions-Policy

8. Improve Error Handling

Issue: Certain invalid requests returned verbose server information.

Recommendation:

- Return generic error messages.
- Disable detailed exception pages.
- Use centralized exception handling with @ControllerAdvice.

9. Continue Using JWT Best Practices

Issue: The assessment confirmed proper JWT validation.

Recommendation:

- Validate signatures.
- Enforce expiration.
- Reject malformed tokens.
- Use HTTPS in production.
- Rotate signing keys periodically.

10. Maintain SQL Injection Protection

Issue: The application successfully resisted SQL Injection attempts.

Recommendation:

- Parameterized queries.

- Prepared statements.
- Input validation.
- ORM best practices.

11. Maintain Cross-Site Scripting Protection

Issue: Stored and Reflected XSS attacks were not successful.

Recommendation:

- Escaping output.
- Validating user input.
- Sanitizing uploaded SVG content.
- Deploying Content Security Policy.

12. Secure API Responses

Recommendation:

- Remove unnecessary attributes.
- Follow the principle of least privilege.
- Return only required fields.
- Avoid exposing internal identifiers when possible.

13. Logging and Monitoring

Recommendation:

- Authentication logging.
- Authorization failure logging.
- Suspicious request detection.
- Security event monitoring.
- Audit trails for sensitive operations.

14. Secure Development Lifecycle

Recommendation:

- Performing code reviews.
- Conducting dependency vulnerability scans.
- Running static code analysis.
- Executing periodic penetration tests.
- Following the OWASP Secure Coding Practices.

15. Production Security Checklist

- Enable HTTPS
- Disable debug mode
- Remove development endpoints
- Disable verbose error messages
- Configure secure HTTP headers

- Enable security logging
- Implement regular security assessments
- Keep dependencies updated
- Apply the principle of least privilege
- Regularly review authentication and authorization mechanisms

Overall Recommendation

The application demonstrates strong implementation of authentication, JWT validation, SQL Injection protection, XSS prevention, and HTTP security headers. However, improvements are required in authorization controls, server-side validation, business logic enforcement, sensitive data handling, and secure API design.

Addressing the identified findings will significantly improve the application's resilience against common web application attacks and align the application more closely with the OWASP Web Security Testing Guide and OWASP Top 10 recommendations.

13. Overall Security Score

A severity-weighted scoring model was used to translate the unique findings of this assessment into a single comparable score, rather than relying on a simple pass/fail percentage that would understate the real-world risk posed by the Critical and High severity findings.

Scoring Methodology

Score = $100 - \Sigma (\text{severity weight} \times \text{number of unique findings at that severity})$, floored at 0.

Severity	Weight	Unique Findings	Subtotal
Critical	12	2	24
High	7	5	35
Medium	3	6	18
Low / Informational	1	3	3
Total Deduction			80

Final Security Score	20 / 100
Risk Grade	F — Critical Risk
Raw Test Pass Rate	64.6% (31/48)

The relatively low Security Score relative to the high raw pass rate is by design: it reflects the fact that 2 Critical and 5 High severity Broken Access Control findings represent a realistic, low-effort path to account compromise and financial abuse, and therefore should dominate the overall risk picture even though the majority of the 48 individual tests confirmed secure behavior.

14. Conclusion

This penetration test exercised 48 distinct test cases against the RideConnect ride-sharing application across reconnaissance, authentication, authorization, input validation, business logic, and secure configuration domains, following the OWASP Web Security Testing Guide and OWASP Top 10 (2021).

The assessment confirmed 16 unique vulnerabilities — 2 Critical, 5 High, 6 Medium, and 3 Low/Informational — concentrated overwhelmingly in OWASP A01 (Broken Access Control). The most significant findings (Rider Identity Tampering and Unauthorized Ride Acceptance) would allow a single authenticated low-privilege user to impersonate other riders and hijack ride transactions belonging to other drivers, representing a realistic and severe business risk if this application were deployed to production without remediation.

Conversely, the assessment also confirmed that several security-critical mechanisms — JWT signature verification and expiration handling, SQL Injection defenses, Cross-Site Scripting output encoding, OTP enforcement, CORS origin validation, and several HTTP security headers — were implemented correctly and resisted all tested attack techniques.

With the access-control remediation outlined in Section 12 applied consistently across all identified endpoints, RideConnect's overall security posture would improve substantially, moving from the current High Risk rating toward a Low-to-Moderate Risk profile suitable for further hardening ahead of any production deployment.

15. Appendix

Appendix A — Tools Used

Tool	Purpose
Burp Suite Community Edition	Intercepting and modifying HTTP requests and responses
Google Chrome	Application testing and browser interaction
Burp Proxy	Capturing HTTP/HTTPS traffic
Burp Repeater	Manual request manipulation and vulnerability verification
Burp Intruder	Authentication and rate-limiting tests
Burp Decoder	JWT decoding and payload analysis
Browser Developer Tools	Client-side inspection and debugging

Appendix B — Testing Environment

Component	Technology
Frontend	React
Backend	Spring Boot
Database	MySQL
Authentication	JWT
File Storage	Local Storage
Testing Environment	Localhost
Frontend URL	http://127.0.0.1:5173
Backend URL	http://localhost:8081

Appendix C — Burp Suite Workflow

Phase 1 — Reconnaissance: Configure Burp Suite Proxy; intercept application traffic; map application endpoints; identify authentication flow.

Phase 2 — Authentication Testing: Login validation; JWT extraction; JWT decoding; token tampering; missing token validation; expired token validation.

Phase 3 — Authorization Testing: IDOR; role validation; resource ownership verification; business logic testing.

Phase 4 — Input Validation: SQL Injection; XSS; Parameter Pollution; HTTP Method Tampering; Mass Assignment.

Phase 5 — Secure Configuration: Security headers; CORS; TRACE; Directory Traversal; Host Header Injection; Cache Control; Error Handling.

Phase 6 — Reporting: Vulnerability verification; severity assessment; evidence collection; remediation recommendations.

Appendix D — Payloads Used

SQL Injection

- ' OR '1'='1
- admin' --
- ' UNION SELECT NULL--
- ' OR 1=1--
- ' AND SLEEP(5)--

Cross-Site Scripting

- <script>alert(1)</script>
-
- <svg/onload=alert(1)>

JWT Testing

- Modified JWT payload
- Expired JWT
- Missing JWT
- Malformed JWT

Parameter Pollution

GET /api/profile/3?role=Driver&role=Admin

Directory Traversal

- ../../../../etc/passwd
- ../../../../Windows/win.ini
- ..%2f..%2f..%2f..%2fetc/passwd

Host Header Injection

- Host: evil.local
- Host: attacker.com

Appendix E — HTTP Status Codes Observed

Status Code	Description
200	Request processed successfully
400	Invalid request
401	Unauthorized
403	Forbidden
404	Resource not found

500	Internal server error
-----	-----------------------

Appendix F — Burp Suite Features Utilized

- Proxy
- Repeater
- Intruder
- Decoder
- HTTP History

Appendix G — Screenshot Index

The following screenshots were captured during the assessment and are embedded inline within Section 10 alongside their corresponding test case.

Test	Description
Test 01	Application Reconnaissance — Landing Page / Traffic Mapping
Test 02	Authentication — Login Request Capture
Test 06	SQL Injection (Authentication) — Payload Rejected
Test 11	Driver Profile IDOR — Cross-Account Profile Leak (incl. password hash)
Test 13	Driver Scheduled Ride IDOR — Cross-Account Ride Data
Test 14	Driver Status Manipulation — Unauthorized Status Update
Test 19	Stored XSS — Payload Injection Test
Test 21	Rider Identity Tampering — Modified Rider/Passenger ID
Test 22	Unauthorized Ride Acceptance — Modified Ride ID
Test 23	OTP Enforcement — Ride Start Blocked Without OTP
Test 24	Fare Manipulation — Client-Controlled Fare Accepted
Test 25	Intruder Brute Force — Login Endpoint Attack Results
Test 31	Mass Assignment — Arbitrary Field Injection & Persistence
Test 32	Privilege Escalation Attempt — Role Field Tampering (Secure)
Test 43	HTTP TRACE Method — Request Rejected
Test 44	Directory Traversal — Payload Rejected with Verbose Error
Test 45	Host Header Injection — Spoofed Host Header Test
Test 47	Error Handling — Verbose Error Disclosure on Invalid Endpoint

Appendix H — References

- OWASP Web Security Testing Guide (WSTG)
- OWASP Top 10 (2021)
- OWASP API Security Top 10 (2023)
- OWASP Secure Coding Practices
- PortSwigger Web Security Academy
- Burp Suite Documentation
- CWE (Common Weakness Enumeration)
- MITRE ATT&CK (Reference)
- Spring Security Documentation
- React Security Best Practices
- JWT RFC 7519

Appendix I — Authorization Statement

This penetration test was performed exclusively against a locally hosted application developed and owned by the author. The assessment was conducted in a controlled environment using authorized accounts and test data. No third-party systems, production services, or external infrastructure were targeted during this assessment. This report is intended solely for educational, research, and professional portfolio purposes.